

TP Grupal: Money Laundering Analysis

Facultad de Ingeniería de la Universidad de Buenos Aires

Sistemas Distribuidos

Cátedra Roca



Integrantes:

Alumno	Padrón
Fernandez, Ignacio Agustín	111019
Molina, Tael Alexis	109458
Serra, Luciano	110694

Índice

Introducción	2
Arquitectura del sistema	7
Tolerancia a fallos	13
Recuperación de los nodos	14
Nodos shardeados y el por qué	14
Glosario y semántica de procesamiento	15
Nodos con estado y nodos “sin estado”	15
Registro, mensaje y batch de mensajes	15
Procesamiento de un batch de mensajes	16
SeqStore y SenderTracker	17
OutputTracker	17
Checkpoint y escritura atómica	18
Sharding	18
Escritura append-only	18
Garantías y límites	19
Tolerancia a fallas: Gateway	19
Rol del componente	19
Modelo de fallas considerado	19
Mecanismos de tolerancia	19
Estado de sesión persistido	19
Casos tolerados	20
Decisiones de diseño y trade-offs	20
Pruebas de performance	20
Tareas realizadas	21

Introducción

Se solicita un sistema distribuido que analice el extracto de transacciones realizadas entre cuentas bancarias en busca de movimientos sospechosos que signifiquen que haya un lavado de dinero de fondo. Para ello se solicita que cada cliente pueda cargar sus datasets y a partir de los mismos recibir el resultado de las 5 consultas solicitadas.

Para la resolución serán empleados los datasets **accounts** y **transactions**. Estos serán el input del cliente y a partir de transformaciones de los mismos se obtendrán los resultados de las 5 consultas.

Se espera que el sistema permita la ejecución aislada y concurrente de distintos clientes. Esto significa que distintos nodos del sistema pueden estar trabajando con información de distintos clientes en el mismo momento. Para garantizar el aislamiento entre clientes se utilizará un *client_id* en todos los mensajes internos evitando leaks. En pos de simplificar la lectura se omitirá este *client_id* en los diagramas.

Esta condición invita a repensar los órdenes de magnitud considerados para el desarrollo del sistema, entendiendo que dados N clientes simultáneos, esos órdenes de magnitud se multiplicarán por ese valor N .

A continuación se visualiza una representación de las transformaciones que presentan los datos en los saltos entre instancias del sistema, así como las dependencias de secuencialidad entre las mismas.

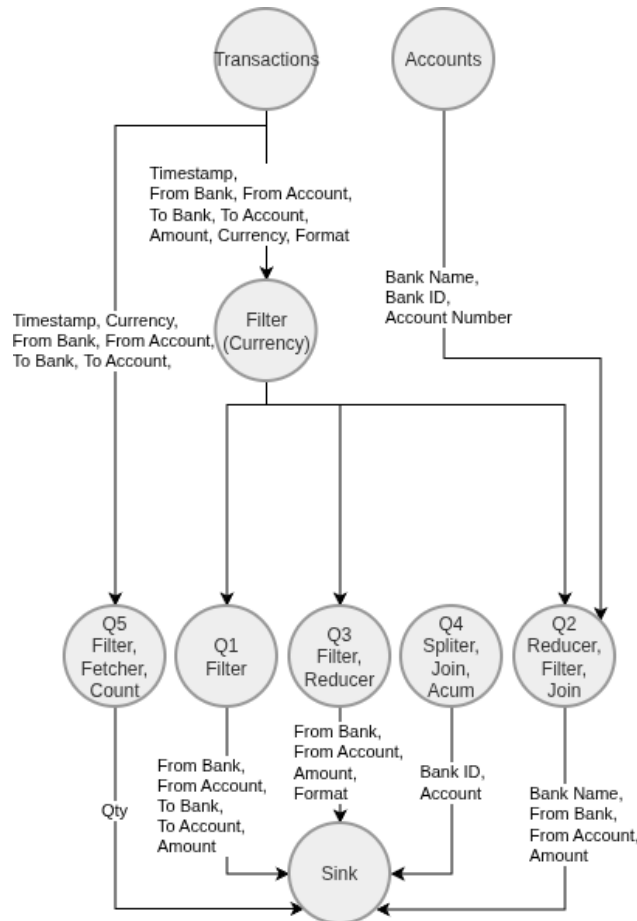


Figure 1: DAG de todo el sistema.

Los siguientes gráficos en detalle la dependencia entre cada instancia.



Figure 2: DAG ejercicio 1.

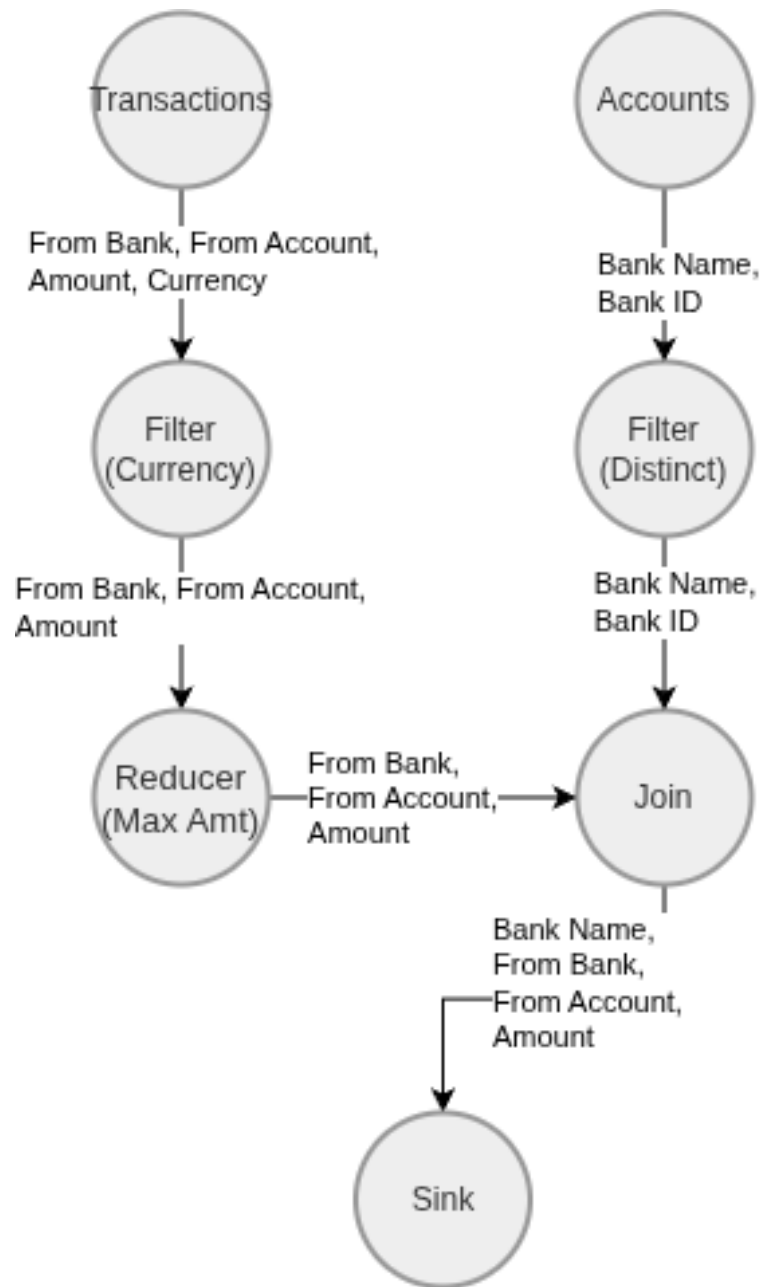


Figure 3: DAG ejercicio 2.

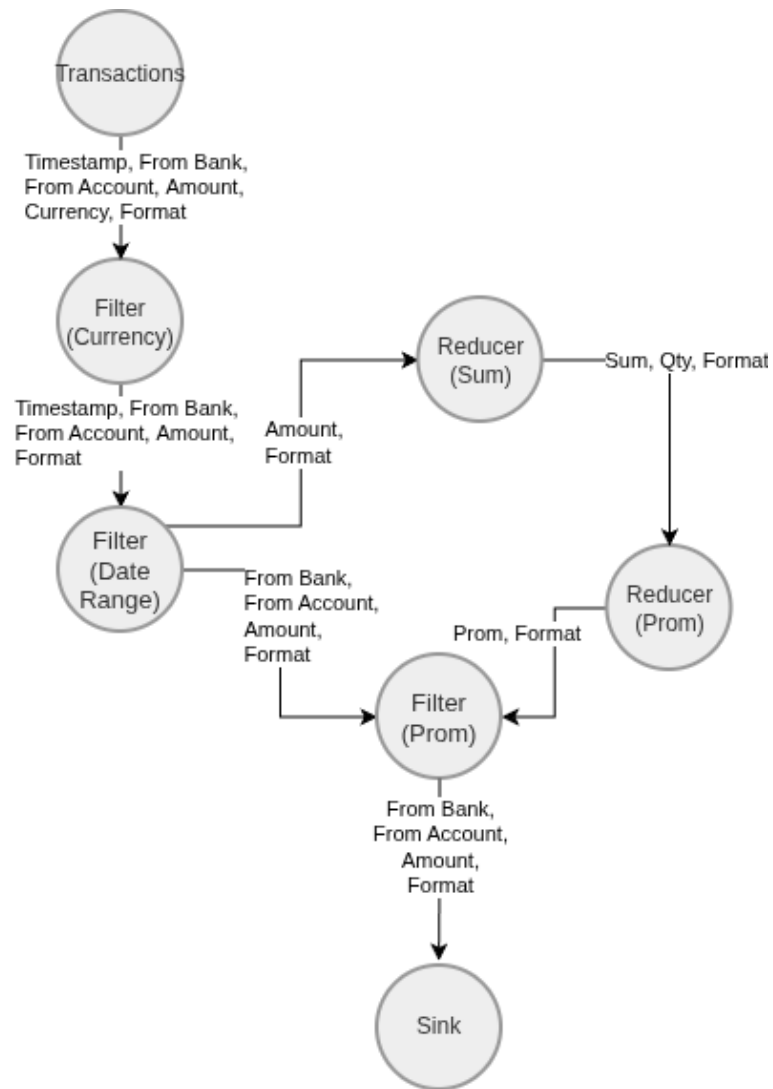


Figure 4: DAG ejercicio 3.

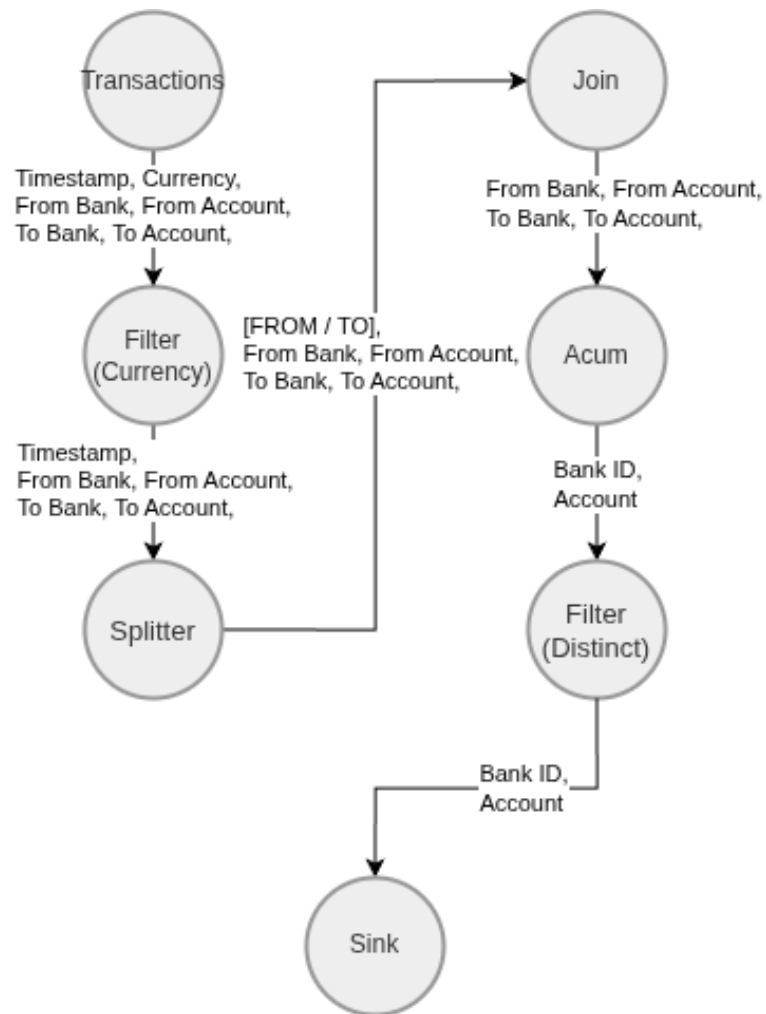


Figure 5: DAG ejercicio 4.

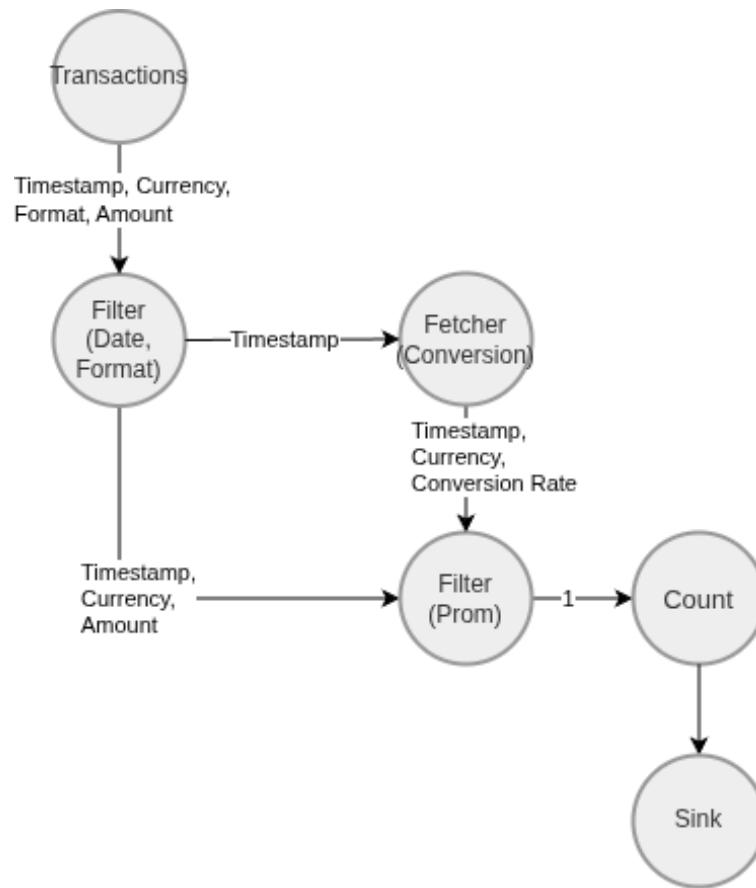


Figure 6: DAG ejercicio 5.

Arquitectura del sistema

Posteriormente, se puede observar una representación de las consultas con un diagrama de robustez para cada una.

Query I

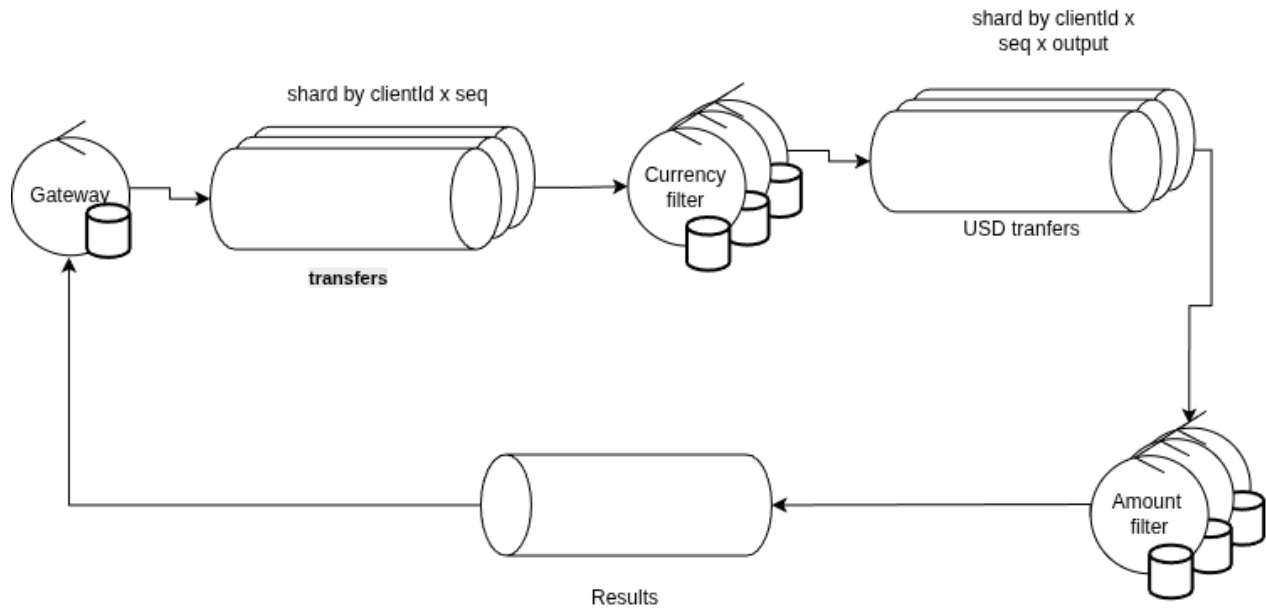


Figure 7: Query I.

Query II

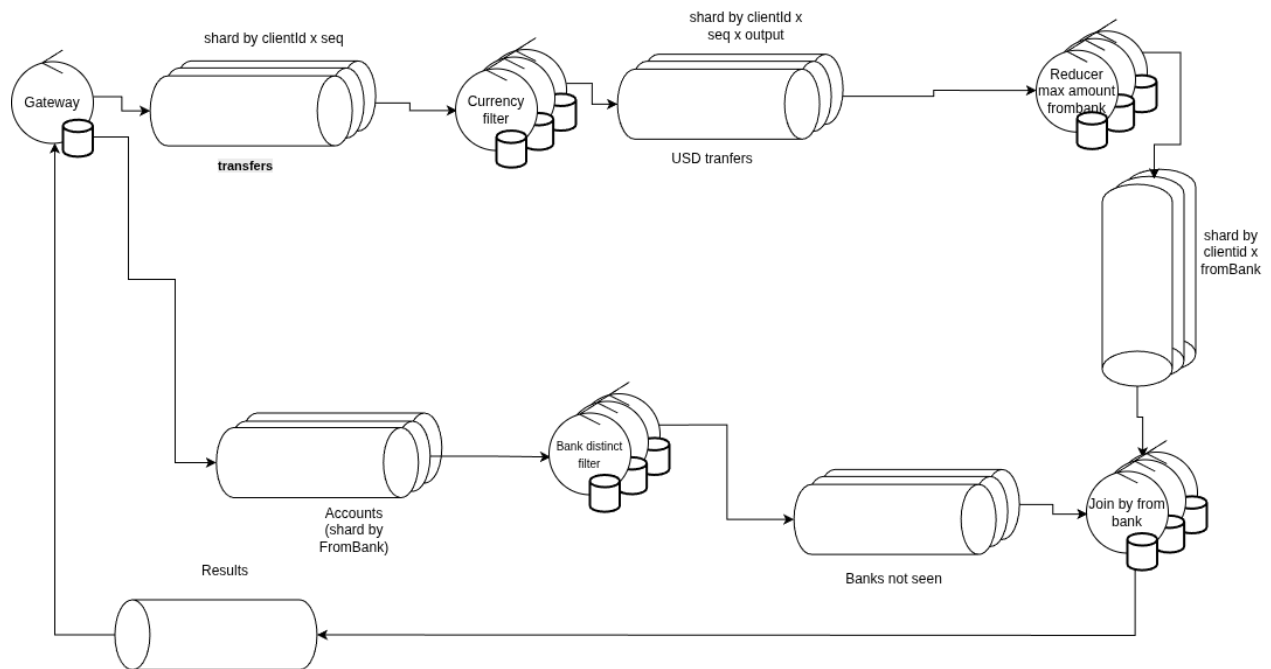


Figure 8: Query II.

Query III

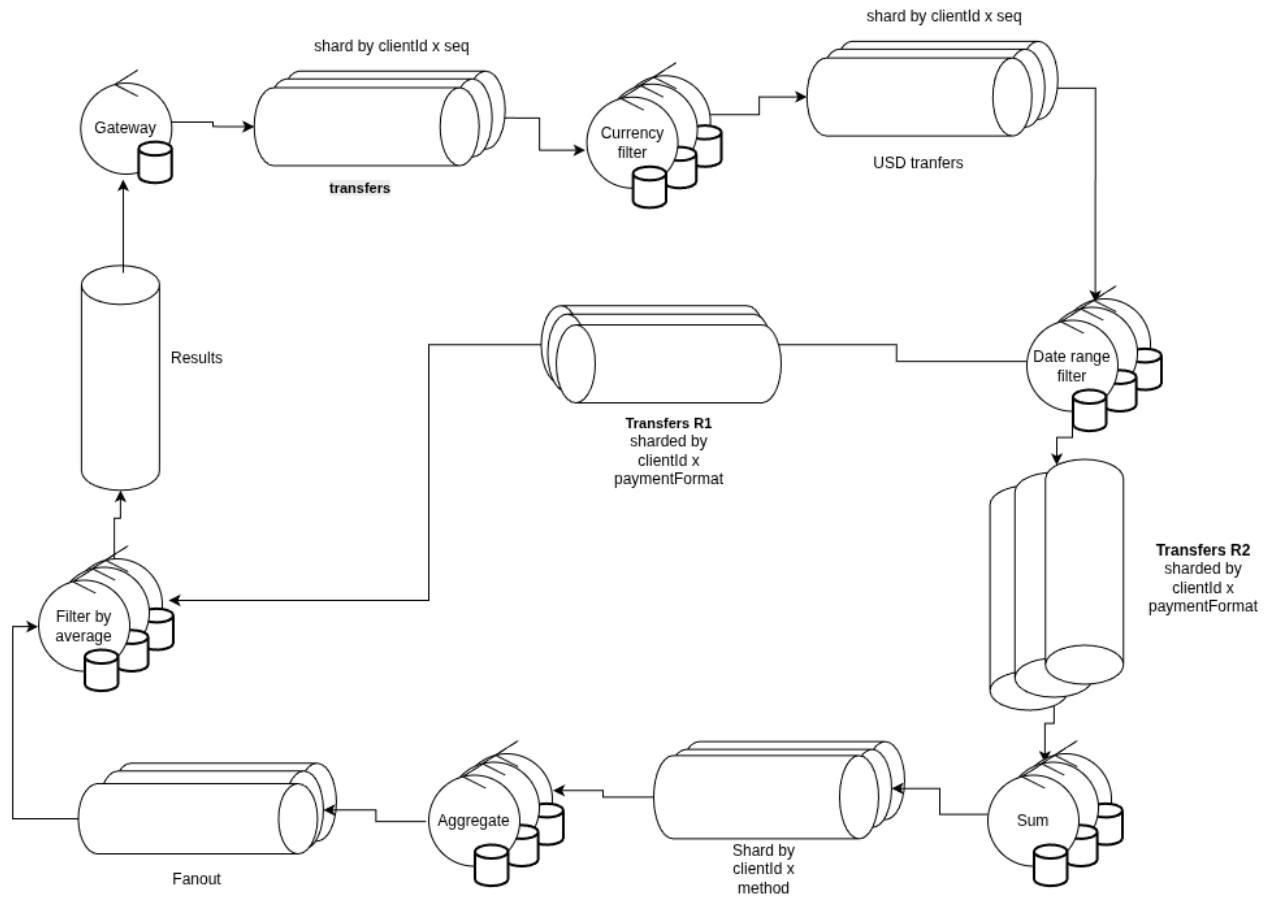


Figure 9: Query III.

Query IV

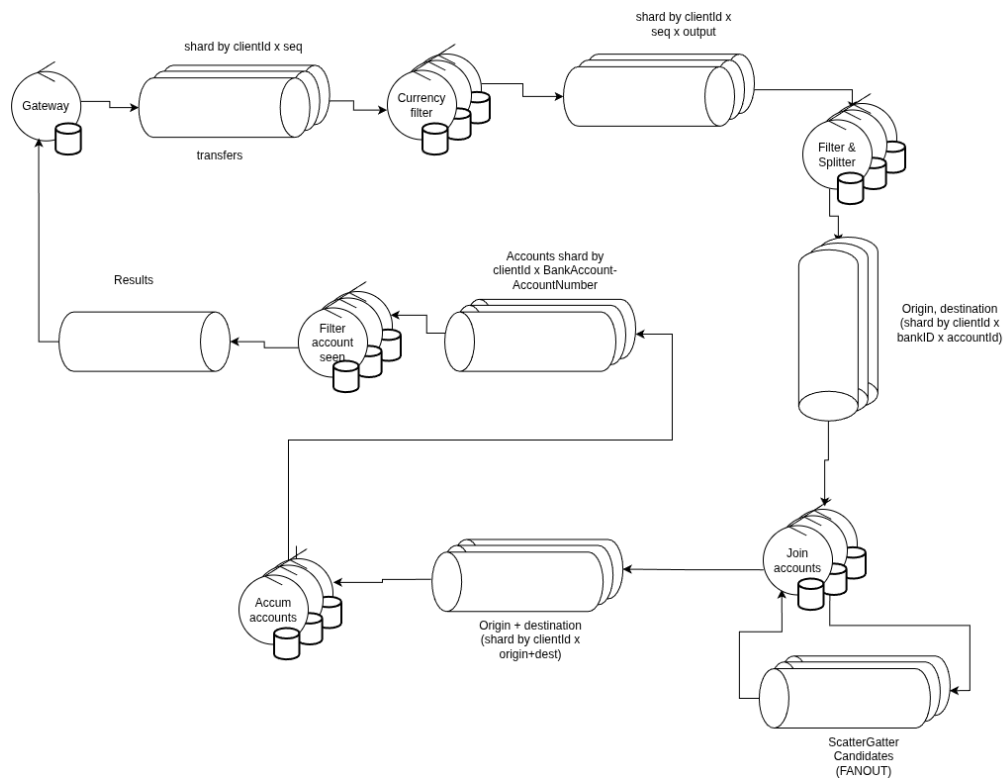


Figure 10: Query IV.

Query V

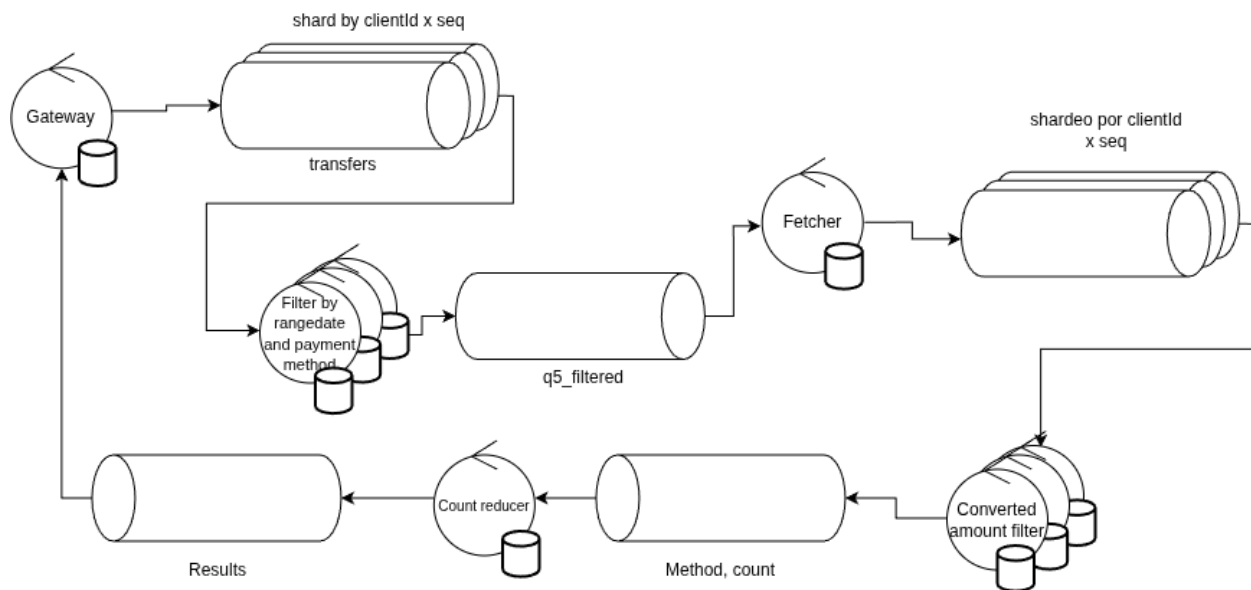


Figure 11: Query V.

Los distintos procesos que ejecutan el pipeline distribuido se pueden observar en el siguiente diagrama.

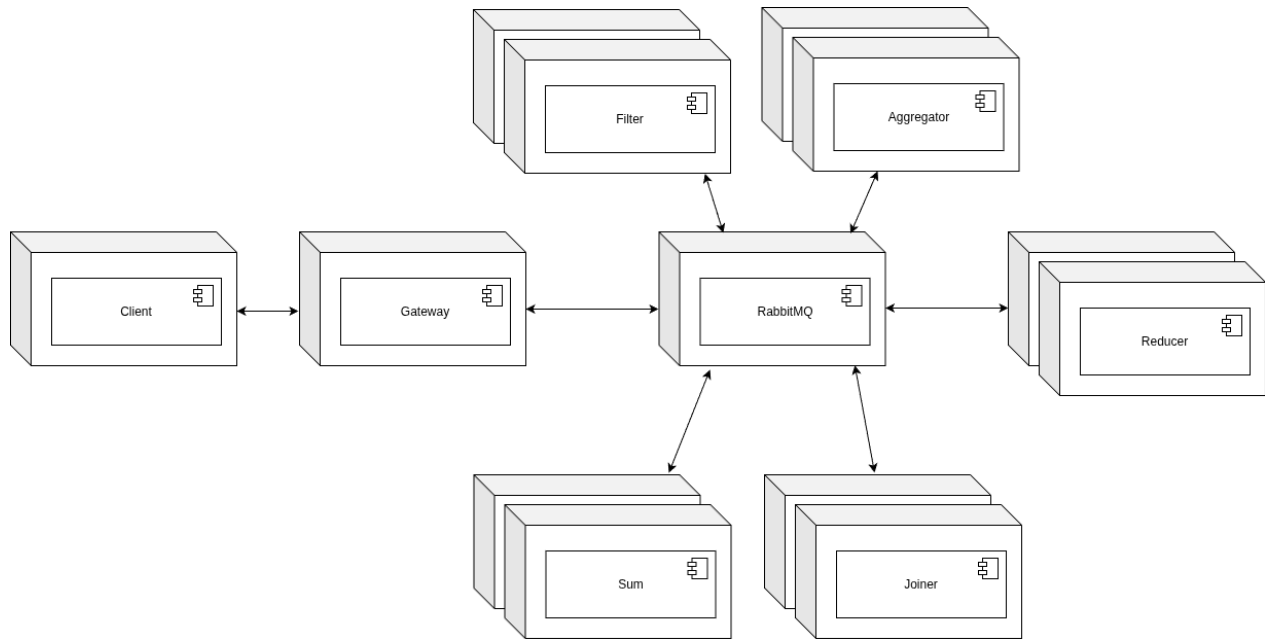


Figure 12: Diagrama de despliegue del sistema.

En él, se puede visualizar cómo se distribuyen los componentes físicos del sistema. Se puede observar que los nodos de procesamiento, como el **Filter**, **Reducer**, **Joiner**, **Aggregator** y **Counter**, se representan con múltiples instancias, indicando que pueden escalar horizontalmente agregando más réplicas según la carga. Todos estos nodos se comunican a través de **RabbitMQ**, que centraliza el flujo de mensajes del sistema. El **Client** y el **Gateway** se despliegan como instancias únicas, siendo este último el encargado de recibir los datos del cliente y distribuirlos hacia las colas correspondientes.

A nivel paquetes de código, se presenta uno llamado **common** compartido por todos los componentes del sistema, el cual contiene los módulos de **Middleware** (wrapper de **RabbitMQ**), **Protocol** (serialización de mensajes) y **Models** (estructuras de datos compartidas).

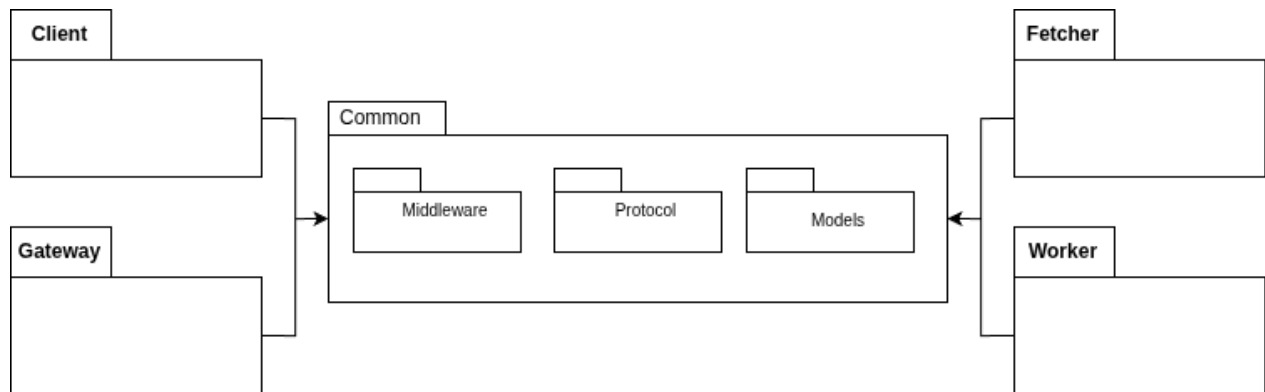
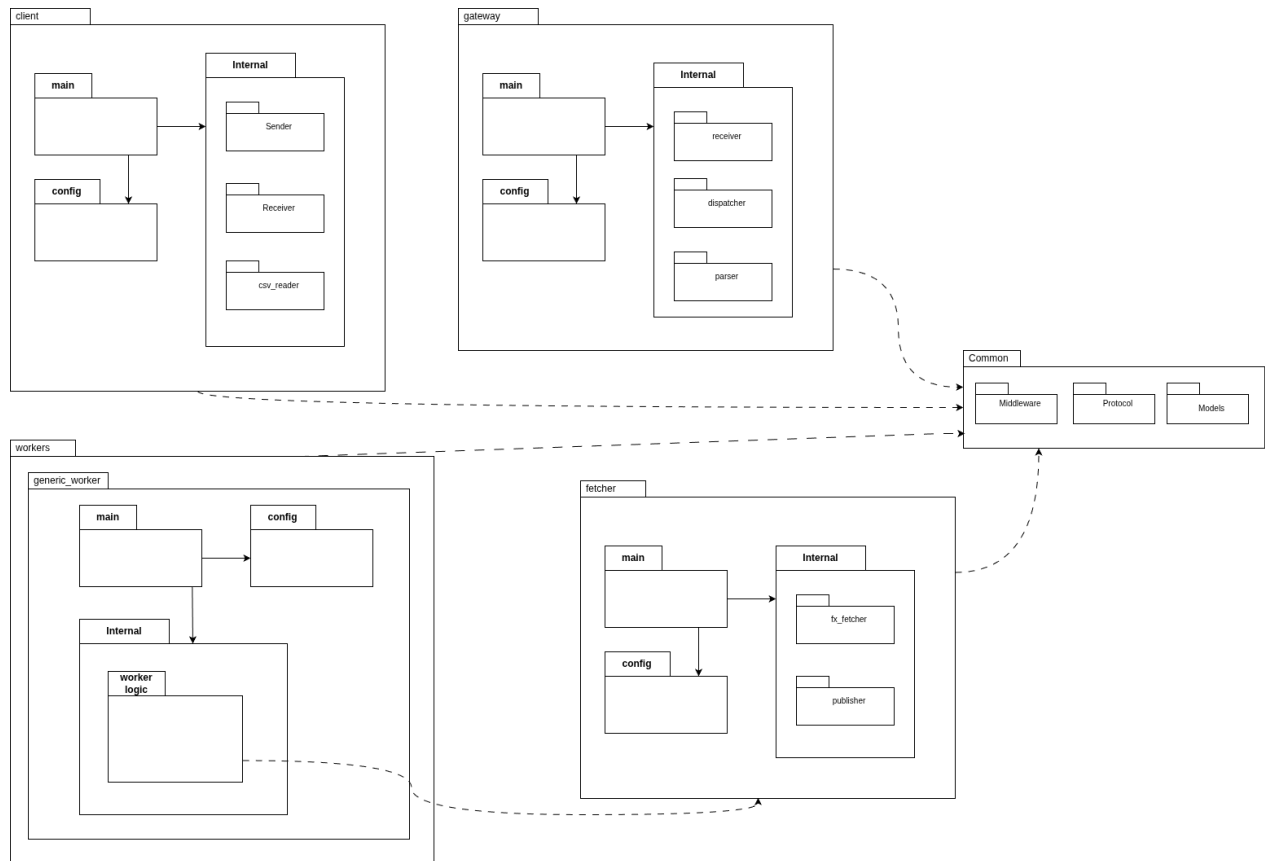


Figure 13: Vista general de dependencias entre paquetes del sistema.

Todos comparten una estructura similar: un módulo **main** como punto de entrada, un módulo **config** para las variables de entorno, y un paquete **internal** con la lógica específica de cada componente. El **Client** contiene los módulos de envío del CSV y recepción del reporte final. El **Gateway** es el punto de entrada del sistema, y se encarga de recibir los datos del cliente y distribuirlos a las colas correspondientes. El **Fetcher** consulta la API externa de cotizaciones y publica los resultados. Finalmente, el **Worker** es una interfaz genérica que expone el ciclo de vida de los mismos, siendo instanciado con distintos comportamientos según

el rol que cumple en el pipeline (filtrado, reducción, join o agregación).



Como caso particular y más interesante para observar, hay ciertos workers que trabajan consumiendo de dos queues a la vez, cuyo life-cycle de forma simplificada se ve de la siguiente forma:

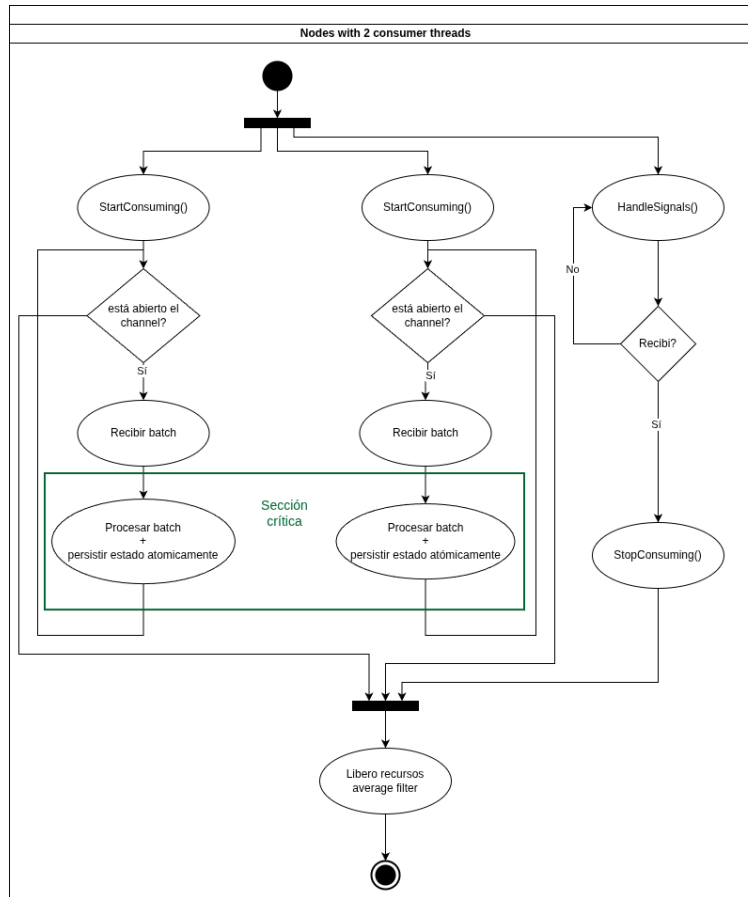


Figure 14: Diagrama de actividades de worker consumidor de dos queues.

Tolerancia a fallos

Se buscó garantizar que el sistema es tolerante a fallos frente a caídas de los nodos workers que viven dentro de él. Como ya es sabido, todos los nodos stateful (reducer, joiner, aggregate) por cada mensaje procesado van actualizando su estado interno, por lo que para garantizar no perderlo, por cada vez que se procesa un mensaje de una queue de RabbitMQ se actualiza su estado interno y se persiste el mismo en un archivo binario (para hacerlo más simplificado y hacer más rápida la lectura y escritura).

La idea además es no reprocesar mensajes ya procesados, para lo que se incorpora el uso de numeros de secuencia para los mensajes de Rabbit. Al haber posibles caídas de los nodos puede llegar a pasar que un nodo llevo a procesar un mensaje y no llegó a guardar que efectivamente procesó al mismo, lo manda a su salida y se cae. En el caso de ser un nodo stateful el hecho de perder ese estado no perjudica en nada porque simplemente se reprocesará ese mensaje y el estado quedará como justo antes de caerse; el problema está en que el mensaje de output potencialmente se enviará dos veces por lo cual si la etapa siguiente no es naturalmente idempotente, puede cambiar el resultado obtenido respecto al esperado. Este tipo de error insertado por la caída de los nodos se maneja con el uso de los números de secuencia ya mencionados en el caso de nodos stateful, en el caso de nodos sin state si bien no va a generar una salida distinta de la esperada porque el gateway también hace este trabajo de los mensajes cuando va armando las respuesta, se busca que cada etapa de procesamiento no tenga que procesar mensajes previamente procesados, para minimizar la carga de trabajo inútil a lo largo de todo el pipeline.

Recuperación de los nodos

Para garantizar que el pipeline está siempre operativo y no cae en situaciones de errores catastróficos, hay un cluster de nodos, llamados *watchdogs* (*perros guardián*) los cuales a través de UDP van enviando heartbeats hacia todos los nodos del pipeline quienes responden a ese mensaje. Como lo importante es saber si el nodo está vivo o no simplemente, UDP es la opción elegida porque otorga un overhead mucho menor en todos los mensajes respecto a TCP, y no tiene el costo del handshake y de los propios heartbeats correspondientes a TCP, los cuales se utilizan para saber que la conexión sigue viva.

Ante la posibilidad de que los nodos *watchdogs* se caigan, se incorpora una elección de líder, utilizando el algoritmo Bully. El líder elegido, no solo es el único responsable de revivir a los nodos una vez dado por muertos, si no que además que es el único que envía los heartbeats hacia los nodos, teniendo una baja carga de trabajo a través de todo el sistema, comparado a la otra opción que sería que todos los *watchdogs* envíen los heartbeats hacia los workers pero que solo el líder reviva a los mismos.

Para la implementación del Bully se utiliza TCP porque es importante tener garantizado el delivery de los mensajes y que sea en orden. Los sockets TCP que se utilizan para comunicar se mantienen abiertos por todo el ciclo de vida de los *watchdogs*; si bien tenerlos abiertos implica un costo, porque como se explicó antes existen los heartbeats y demás, abrir y cerrar sockets indiscriminadamente es un costo muy alto, y es más complicado de por sí. Si se garantizara la correcta distribución de los nodos *watchdogs* podrían abrirse y cerrarse los mismos, porque ese costo estaría amortizado pero de por sí casi seguro que implicaría una latencia mayor y otros costos que implica tener nodos comunicandose en distintas partes del mundo.

Cada vez que un *watchdog* se levanta abre una comunicación con los otros de su mismo cluster que tengan ID mayor. Para garantizar que todos están comunicados con todos, al revivir a uno de ellos el líder le avisa al resto del cluster que acaba de revivir a ese nodo, para en el caso de que $ID_{nodo-vivo} < ID_{nodo-recien-levantado}$ pueda el ya vivo crear ese canal de comunicación con el recién levantado.

Nodos shardeados y el por qué

Para implementar la tolerancia a fallos el esquema elegido fue utilizar shardeo en las etapas de procesamiento porque es mucho más sencillo de implementar el reprocesamiento de los mensajes. Para ello se ilustra un ejemplo para qué se entienda mejor.

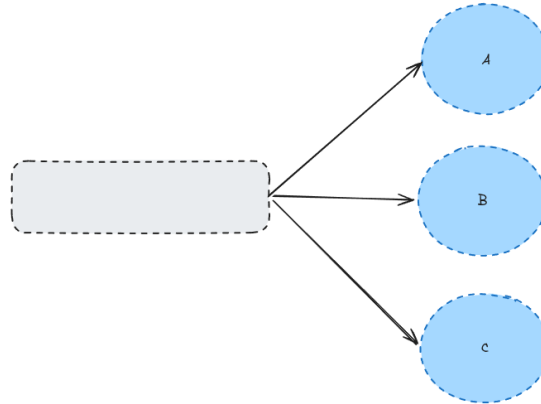


Figure 15: Worker cluster compartiendo una working queue

En este caso, si el nodo A procesa un mensaje y se cae antes de persistir que procesó dicho mensaje, pero post persistir el estado interno y se cae, por el NACK de Rabbit ese mensaje va a ser reencolado en la working queue que comparten los workers de este clúster, provocando que potencialmente ese mensaje sea reprocesado por el nodo B o el nodo C, cambiando el estado interno de quien sea que tome ese mensaje, generando resultados erróneos respecto a una ejecución serial del pipeline, ya que en afectó el estado interno de A un mensaje X y también afectó el mismo mensaje X el estado de B/C.

Una solución superadora a esta arquitectura es en los lugares donde sea apropiado (por ejemplo en el clúster de los *currency filter*) utilizar una working queue junto a un nodo que se encargue del procesamiento centralizado de mensajes ya procesados, ya que no hay una razón intrínseca del problema que requiera el uso de un sharding cluster. Para implementar dicha solución la arquitectura de esta sección se vería algo como lo siguiente:

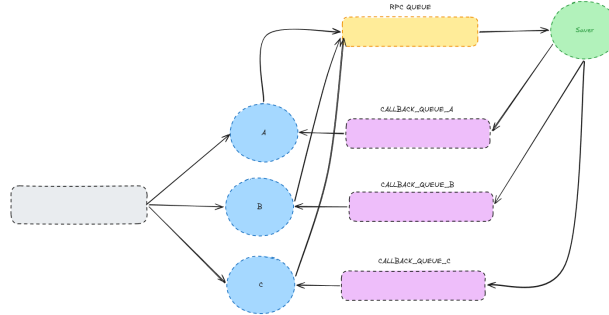


Figure 16: Arquitectura ideal

Donde el nodo saver utilice un algoritmo de coordinación y acuerdo como el Two Phase Commit, para comunicarles a los nodos que cierto mensaje *X* ya fue procesado exitosamente o que no debe ser reprocesado porque otro nodo del mismo clúster ya lo hizo.

Glosario y semántica de procesamiento

Nodos con estado y nodos “sin estado”

El sistema distingue principalmente dos familias de nodos de procesamiento: nodos con estado (*stateful*) y nodos sin estado (*stateless*). La diferencia no es si guardan bytes en memoria o disco, sino si la lógica de negocio necesita acumular información entre mensajes para producir el resultado.

Un nodo con estado mantiene información propia del cálculo de la consulta. Por ejemplo, un **Reducer** de la query 2 conserva, por cliente, el máximo visto por banco hasta recibir todos los mensajes esperados; un **Join** mantiene buffers de ambos lados del join hasta poder cruzarlos; y otros acumuladores guardan sumas, conteos o estructuras intermedias. Si el nodo cae, ese estado de negocio debe recuperarse para no volver a empezar el cálculo desde cero.

Un nodo “sin estado”, en cambio, no necesita recordar registros anteriores para decidir qué hacer con el registro actual. Un filtro puede leer un batch, aplicar una condición y emitir los registros que pasan. Sin embargo, en esta implementación estos nodos no son completamente *stateless*, aunque se toma ese naming por cuestiones prácticas: sí mantienen un estado operacional mínimo. Ese estado no pertenece al cálculo de negocio, sino a la robustez distribuida: qué mensajes ya fueron procesados, cuántos batches salieron por cada destino y qué EOFs llegaron. Por eso los filtros tienen **SenderTracker**, **OutputTracker** y **Checkpoint**, aunque su transformación principal sea *stateless*.

Ambos tipos de nodo mantienen estado en memoria y lo persisten por cliente. En los nodos *stateless* se persiste principalmente metadata de control: secuencias ya procesadas, EOFs recibidos y cantidades emitidas por destino. En los *stateful* se persiste esa misma metadata operacional y, además, el acumulado propio de la consulta: máximos parciales, conteos, sumas, buffers de join o registros necesarios para completar una etapa posterior.

Registro, mensaje y batch de mensajes

Un registro es la unidad lógica de datos del dominio: una cuenta, una transferencia, un resultado de una query, etc. Cada tipo de registro tiene un codec binario en el paquete `messageprotocol/rabbit/records` o en los canales específicos, cuya finalidad es definir como se serializa y deserializa ese tipo de dato.

Un mensaje de RabbitMQ transporta un batch de registros. El paquete `messageprotocol/rabbit/batch` serializa cada mensaje con un encabezado común: `clientID`, `queryID`, tipo de mensaje, `senderID` y `seq`.

Luego agrega la cantidad de registros y el payload binario. Además del tipo batch existen mensajes EOF, que anuncian cuántos batches se enviaron desde un sender, y mensajes abort, que cancelan el procesamiento de un cliente y limpian su estado en el nodo. Un batch de mensajes es el grupo de mensajes de RabbitMQ que un consumidor toma antes de ejecutar su lógica y hacer ack. El middleware implementa **StartConsumingBatch**: junta mensajes hasta alcanzar **batchSize** o hasta que venza **flushInterval**. En ese punto delega al nodo con **[]Message** y confirma o rechaza todos los deliveries juntos usando ack/nack múltiple.

Procesar batches de mensajes optimiza el sistema porque amortiza costos fijos: menos acks individuales, menos checkpoints, menos escrituras a disco y más oportunidades de agrupar salidas. A la vez, el límite por cantidad y por tiempo evita esperar indefinidamente cuando la carga es baja.

Procesamiento de un batch de mensajes

El procesamiento interno de un nodo puede entenderse como un *batch de batches*. Cada mensaje de RabbitMQ contiene un batch de registros; a su vez, el consumidor recibe un grupo de mensajes y los procesa dentro de una misma invocación. El **FilterAndSplitter** es un ejemplo representativo de este patrón.

Durante el procesamiento, cada mensaje se maneja de manera individual, pero los efectos sobre persistencia y ACK se diferencian hasta terminar el batch completo de mensajes. Esto permite actualizar el estado en memoria tantas veces como sea necesario y persistir una sola vez por cliente involucrado.

```
1 handleBatch(mensajes, ack, nack):
2     modificados = {}
3     completados = {}
4     abortados = {}
5
6     for mensaje in mensajes:
7         input = decodificar(mensaje)
8         clientID = input.clientID
9         estado = estadoEnMemoria(clientID)
10        tracker = estado.senderTracker
11
12        if input.esAbort() or abortados[clientID]:
13            abortados[clientID] = true
14            modificados[clientID] = estado
15            continue
16
17        if tracker.esDuplicado(input.senderID, input.seq):
18            continue
19
20        if input.esEOF():
21            tracker.registrarEOF(input.senderID, input.total, input.seq)
22        else:
23            tracker.registrarBatch(input.senderID)
24            processBatch(input, estado)
25
26        tracker.claim(input.senderID, input.seq)
27        modificados[clientID] = estado
28
29        if tracker.estaCompleto(cantidadDeSendersEsperados):
30            finish(clientID, estado)
31            completados[clientID] = true
32
33    for clientID, estado in modificados:
34        if abortados[clientID]:
35            propagarAbort(clientID)
36            eliminarEstadoEnMemoria(clientID)
37            eliminarCheckpoint(clientID)
38            continue
39
40        if completados[clientID]:
41            eliminarEstadoEnMemoria(clientID)
42            eliminarCheckpoint(clientID)
43            continue
44
45    guardarCheckpoint(clientID, estado)
46
```

Listing 1: Pseudocódigo del procesamiento de un batch de mensajes

En nodos stateless, la función `processBatch` no acumula estado de negocio: transforma o filtra registros y genera salidas. En el caso del `FilterAndSplitter`, cada transferencia que pasa el filtro temporal se proyecta al formato de la query 4, se duplica en dos partes lógicas y se envía al shard correspondiente. Como el nodo no está creando una nueva etapa de ordenamiento lógico, las salidas reutilizan la secuencia `seq` recibida en el mensaje de entrada. Esa secuencia, junto con el `senderID`, permite que el consumidor siguiente detecte redeliveries o duplicados.

Después de procesar cada mensaje no se persiste inmediatamente. El estado queda actualizado en memoria y el cliente queda marcado como modificado. Al terminar de recorrer todos los mensajes recibidos, el nodo decide qué hacer por cliente: si hubo abort, limpia estado y propaga el abort; si el cliente completó la etapa, ejecuta la función de cierre, elimina el estado y borra el checkpoint; si sólo hubo avance parcial, guarda un único checkpoint con el estado resultante. De esta forma, un batch de mensajes que contiene varios mensajes de un mismo cliente produce una sola escritura durable para ese cliente.

El ACK se emite únicamente cuando todas las operaciones del batch terminaron correctamente. Como RabbitMQ permite usar ACK múltiple, el nodo confirma de una vez todos los mensajes entregados en ese batch. Si ocurre un error de procesamiento, envío o persistencia, el nodo ejecuta `nack`, detiene el consumo y permite que RabbitMQ redespache los mensajes. En ese escenario, los `SenderTracker` recuperados desde checkpoint evitan reprocesar secuencias ya reclamadas.

SeqStore y SenderTracker

El `SeqStore` es una estructura que se utiliza para deduplicar mensajes recibidos. Internamente usa un bitmap: cada `seq` se representa con un bit. Esto permite consultar y reclamar secuencias en memoria con bajo costo, y serializar sólo el prefijo relevante para persistirlo. Esta estructura resuelve la problemática de poder identificar que mensajes ya fueron procesados en un sistema que no ofrece garantía de orden de procesamiento. En lugar de contar con un mapa de números de secuencia ya procesados, se cuenta con un mapa de bits donde el número de secuencia determina el offset para acceder al bit que lo representa.

El `SenderTracker` compone varios `SeqStore`, uno por sender de la capa anterior. Para cada sender guarda: los `seq` ya reclamados, cuántos batches se procesaron, si llegó EOF y cuántos batches se esperaban según ese EOF. Con esa información un nodo puede detectar duplicados, saber cuándo recibió todo lo que debía recibir y recién entonces emitir su propio EOF hacia la siguiente etapa.

Este mecanismo apunta a garantizar idempotencia de procesamiento. Si Rabbit reentrega un mensaje porque el consumidor cayó antes del ack, el nodo puede ver que el par (`senderID`, `seq`) ya fue procesado y no volver a aplicar la lógica de negocio.

OutputTracker

El `OutputTracker` registra cuántos batches emitió un nodo por routing key o destino lógico. Esto es necesario porque los EOF no indican cuántos registros se produjeron, sino cuántos batches debe esperar el siguiente nodo de un sender. Por ejemplo, un filtro puede recibir un batch con registros, filtrar todos y no emitir nada; o puede dividir la salida entre distintos shards. Al finalizar, el nodo consulta el `OutputTracker` y envía un EOF a cada destino con el total exacto de batches publicados para ese destino.

En el gateway aparece el mismo patrón para transferencias: al shardear y reenviar los batches de entrada, se cuenta cuántos batches fueron enviados a cada shard para poder emitir los EOF correctos al final de la fase.

Por ejemplo, si un nodo recibe tres batches de un sender y, luego de filtrar, emite dos batches al `shard-0`, ninguno al `shard-1` y uno al `shard-2`, el `OutputTracker` conserva esos totales. Al finalizar, el nodo envía EOF a los tres destinos: `total=2`, `total=0` y `total=1`. De esta forma, cada consumidor sabe exactamente cuántos batches debe esperar antes de considerar completa su entrada.

Checkpoint y escritura atómica

El **Checkpoint** persiste el estado por cliente en archivos `.bin`. La escritura se hace de forma atómica: primero se serializa el estado, se escribe en un archivo temporal `.tmp`, se fuerza a disco con `fsync`, se cierra el archivo y finalmente se hace `rename` al path definitivo. Este patrón evita dejar un checkpoint parcialmente escrito como versión válida si el proceso cae durante la escritura.

Los nodos guardan checkpoint después de procesar un batch de mensajes y antes de confirmar el `ack`. Si la persistencia falla, el nodo hace `nack` y detiene el consumo. Si el proceso cae antes del `ack`, Rabbit puede reentregar los mensajes; al recuperar desde el checkpoint, el **SenderTracker** permite ignorar lo que ya fue reclamado.

El gateway usa una variante adicional para la sesión del cliente: guarda fase y `confirmedSeq`. De esa forma, si el cliente se desconecta y vuelve con una sesión conocida, el gateway puede responder desde qué fase y secuencia retomar.

Sharding

El sharding se implementa con routing keys del estilo `shard-N`. Cada cluster conoce cuántos shards tiene y usa un hash FNV para mapear `clientID` más una o más claves de dominio a un índice. Incluir el `clientID` evita mezclar clientes distintos y permite que cada ejecución se procese de manera independiente.

La clave elegida depende de la etapa. En cuentas se usa el banco, en transferencias de entrada se usa la secuencia del batch para repartir la carga y en nodos posteriores se usan claves de dominio provistas por cada transformación. La regla general es que los registros que deben encontrarse en una misma etapa stateful se mandan al mismo shard.

Escritura append-only

El último nodo de la query 2 tiene una peculiaridad: Debe mantener un alto volumen de transacciones en espera hasta que se complete el cálculo del promedio del otro rango de fechas. Esto implica tener que recibir esas transacciones durante más que un momento efímero. A su vez, por la tolerancia a fallos estas transacciones que ahora forman parte de su estado interno deben persistirse. La solución diseñada implica separar el estado interno en 2. Por un lado el manejo de checkpoints que se menciona en secciones anteriores, donde el nodo guarda todo su estado interno exceptuando el listado de transacciones, y por otro, un log de transacciones. La dificultad técnica radica no solo en la consistencia de estados (sumando un estado **ARCHIVO 1 PERSISTIDO**, **ARCHIVO 2 NO PERSISTIDO**) ante caídas, sino también en como garantizar la correcta escritura del log sin reescribirlo entero ni levantarlo en memoria (considerando el volumen). Se optó por el uso de `appendlog`. En este se realiza una escritura por batch de mensajes procesados, haciendo `append` al final del archivo. Ante fallas puede quedar un archivo "corrupto", a diferencia de las escrituras atómicas. Como la escritura se hace sobre el final del archivo, al reiniciar el nodo va a recorrer el archivo completo con el objetivo de validar con el checksum de la última entradas si la misma está completa, y caso contrario eliminarla. Estas entradas son llamadas frames, cada uno con un `senderID`, `seq`, cantidad de registros, payload y checksum CRC32.

La ventaja es que no hace falta reescribir todo el estado acumulado ante cada mensaje: se agregan entradas al final y, en recuperación, se reconstruye el estado leyendo el log. Cuando se combina con **SenderTracker**, la lectura puede ignorar entradas duplicadas por (`senderID`, `seq`).

El **AverageFilter** de la query 3 muestra la separación entre ambos mecanismos. El checkpoint se usa de la forma habitual para persistir el estado in-memory del cliente dentro del nodo: trackers de entrada y datos auxiliares necesarios para saber si ya llegaron todos los mensajes. Las transacciones pendientes, en cambio, no se guardan en el checkpoint. Se escriben en un archivo `append-only` porque se procesan recién cuando el nodo ya recibió toda la información necesaria. Si esas transacciones se guardaran dentro del checkpoint, cada nuevo batch obligaría a reescribir el acumulado completo; con `append-only` sólo se agregan las entradas nuevas al final del archivo.

Al finalizar, el nodo reabre el `append log`, itera las transacciones descartando repetidas y genera la salida. Luego elimina el log correspondiente al cliente. De esta manera el checkpoint conserva el estado de control del cliente, mientras que el `append log` funciona como almacenamiento durable y eficiente para registros que esperan procesamiento diferido.

Garantías y límites

El sistema busca garantizar que cada batch de entrada sea procesado una sola vez a nivel lógico por cada nodo, incluso ante redelivery de RabbitMQ. Esa garantía se apoya en `senderID`, `seq`, `SenderTracker`, checkpoints y acks manuales.

No se garantiza deduplicación del output generado por cada nodo, pero si que el nodo que recibe un input lo deduplique localmente, y por, el shardeo realizado, el mismo mensaje siempre lo procesa el mismo nodo.

Gateway

Rol del componente

El gateway es el único punto de entrada y salida del sistema, ya que es la frontera entre el cliente, que se comunica por TCP, y el pipeline de procesamiento, que se comunica por RabbitMQ. Recibe los lotes de cuentas y transferencias del cliente, los descompone y los envía hacia el pipeline, y en el sentido inverso consume los resultados que el pipeline deposita en la cola de resultados y se los entrega al cliente.

Por este rol de frontera, el gateway concentra el estado de sesión de cada cliente y es el responsable de que una caída propia o del cliente no produzca resultados inconsistentes.

Modelo de fallas considerado

El sistema acepta fallas de tipo *crash-stop*: un proceso puede caerse en cualquier momento y, al reiniciar, no conserva nada de su estado en memoria, sólo lo que haya persistido en disco.

La distinción central que guía todo el diseño es **proceso vivo vs. proceso muerto**:

- Un cliente cuyo **proceso sigue vivo** (corte de red, o el gateway que rebotó y volvió) conserva su `sessionID` en memoria. Su lazo de reconexión reabre la conexión, presenta ese mismo identificador y reanuda la sesión.
- Un cliente cuyo **proceso murió** y reinició perdió el `sessionID` de memoria. Al reconectar se presenta sin identificador y el gateway lo trata como un cliente nuevo.

El gateway no necesita distinguir ambos casos en el momento de la desconexión: simplemente arranca un temporizador de muerte y deja que el *reaper* decida.

Mecanismos de tolerancia

Estado de sesión persistido

Por cada cliente, el gateway mantiene en disco `sessionstore` la siguiente información, escrita de forma atómica `diskstore.WriteAtomic`:

- La **fase** de la sesión (`Accounts`, `Transfers`, `Results`).
- Los **seq confirmados** por fase: hasta qué número de batch la ingesta fue persistida con éxito.
- Los **trackers de deduplicación** de resultados por query, que registran qué resultados del pipeline ya fueron entregados y los EOF recibidos de cada sender.
- Las **queries ya reportadas** al cliente, para no reenviar el EOF de una query terminada.
- Los **aborts pendientes** y el contador de `nextClientID`.

Casos tolerados

Escenario	Comportamiento
Cae el gateway, el cliente sigue vivo	Al reiniciar, el gateway recupera el <code>sessionstore</code> desde disco. El cliente reconecta con su <code>sessionID</code> en memoria, reanuda la ingesta desde <code>resumeSeq</code> y recibe el buffer de resultados pendientes. Resultado final consistente.
Corte transitorio de red (proceso del cliente vivo)	El lazo de reconexión reabre la conexión con el mismo <code>sessionID</code> dentro de <code>ClientTimeout</code> y reanuda. Es el mismo mecanismo que el caso anterior.
Cae un nodo del pipeline y reprocesa resultados	La deduplicación por (<code>queryID</code> , <code>senderID</code> , <code>seq</code>) descarta los reenvíos; el cliente nunca ve resultados duplicados.
Caída del gateway entre el <code>ack</code> y la entrega	El <code>ack</code> a RabbitMQ se emite después de persistir el estado; el resultado se reentrega tras el reinicio. No hay pérdida.
Cae el cliente (proceso muerto) con el gateway vivo	El <i>reaper</i> declara muerta la sesión vieja tras <code>ClientTimeout</code> , libera recursos y aborta el pipeline. El cliente reiniciado entra como uno nuevo.

Decisiones de diseño y trade-offs

- **Frecuencia de checkpoint** (`SeqCheckpointEvery`). Avanzar `confirmedSeq` con menor frecuencia reduce los `fsync` y mejora el throughput de ingesta, a costa de reprocesar más lotes ante una caída. Es una palanca de durabilidad vs. rendimiento.
- **`fsync` agrupado por lote** (`ResultsBatchFlushInterval`). Agrupar la persistencia del estado de sesión por lote en lugar de por mensaje redujo drásticamente el costo de drenado de resultados. El intervalo regula latencia (qué tan “en vivo” ve el cliente los resultados) vs. rendimiento.
- **Escritura atómica del estado.** Toda persistencia del `sessionstore` usa escritura a temporal + `rename`, de modo que un corte a mitad de escritura nunca deja el estado en un punto intermedio inválido.
- **No tolerancia a caídas de un cliente** Toda persistencia del `sessionstore` usa escritura a temporal + `rename`, de modo que un corte a mitad de escritura nunca deja el estado en un punto intermedio inválido.
- **No tolerar la caída del proceso cliente.** Se evaluó persistir el `sessionID` y el progreso de salida del lado del cliente para que un cliente reiniciado pudiera retomar su sesión, pero fue descartado por la siguiente razón: sostener esa reanudación implicaba bastante más código y casos de borde difíciles (persistencia y limpieza del `sessionID` en disco, escritura idempotente de los archivos de salida en modo *append*, deduplicación local de resultados ya escritos, y la coordinación de todo eso con el reinicio del proceso), a cambio de un beneficio marginal. Se optó por mantener el cliente simple: el `sessionID` vive sólo en memoria. La resiliencia ante cortes transitorios de red —donde el proceso del cliente sigue vivo— se conserva igual, porque depende de ese identificador en memoria y no del disco; lo único que no se reanuda es un proceso que efectivamente murió y reinició.

Pruebas de performance

Se ejecutaron cuatro pruebas sobre distintas configuraciones del sistema. En todos los casos se contó con **2 instancias de cada worker**.

Dataset	Clientes	Tolerancia	Chaos	Parámetros chaos	Duración
Large	1	✓	–	–	4 min 11 s
Large	1	×	–	–	1 min 36 s
Medium	2	–	×	–	5 min 26 s
Medium	2	–	✓	ROUNDS=15, SLEEP_ROUND=7, CONTAINERS.RESTART=3	6 min 38 s

Table 1: Tiempos de ejecución según configuración (2 instancias por worker)

Análisis

Los resultados muestran que la tolerancia a fallos introduce un overhead significativo: con *dataset large* y 1 cliente, habilitarla incrementó el tiempo de ejecución de **1 min 36 s** a **4 min 11 s**, es decir, un aumento de aproximadamente el **163%**. Esto es esperable dado el costo asociado a los mecanismos de replicación y recuperación.

Por otro lado, la inyección de caos sobre el *dataset medium* con 2 clientes produjo un impacto considerablemente menor: el tiempo pasó de **5 min 26 s** a **6 min 38 s**, un incremento del **22%**. Esto sugiere que el sistema logra absorber las fallas y recuperarse sin degradar severamente la performance general, lo cual valida el correcto funcionamiento del mecanismo de tolerancia ante caídas de contenedores.

Tareas realizadas

Tarea	Responsable
Client + Protocol	Fernandez
Gateway	Serra
Middleware(s)	Serra
Q1	Molina
Q2	Serra
Q3	Serra y Fernandez
Q4	Fernandez
Q5	Molina
Tests Happy Path	Serra, Molina
Docker compose	Fernandez